

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА − Российский технологический университет»

РТУ МИРЭА

Практическое занятие № 10

по дисциплине «Технологии разработки приложений на базе фреймворков»

Тема работы: Система управления медицинскими записями

	студент группы		
Выполнил:	ЭФБО-01-24		Осипова В.А.
	(группа)	*(подпись)*	*(Фамилия И.О.)*
Принял:	доц.		Ящун Т.В.
	(должность)	*(подпись)*	*(Фамилия И.О.)*

Москва
2026

Цель работы

Освоить возможности административной панели Django (настройка отображения, поиска, фильтрации и связанных объектов) и углубить навыки работы с данными через Django ORM. Применить полученные знания в индивидуальном сквозном проекте «MedicalSystemIP» (Система управления медицинскими записями).

Настройка административной панели:
В файлах admin.py для приложений patients, specialists и appointmentsнастроены классы ModelAdmin.

- Для модели Appointment настроены list_display (вывод пациента, врача, даты и статуса), search_fields (поиск по связанным полям через patient__last_name и specialist__last_name), list_filter (фильтрация по статусу и дате).
- В карточку пациента добавлен TabularInline для отображения связанных приемов (модель Appointment).
- В админку приемов добавлены массовые действия для изменения статусов (mark_as_completed, mark_as_cancelled).

Изменение моделей и миграции:

1. В модель Appointment внедрено поле status на основе models.TextChoices(Запланировано, Завершено, Отменено). Создана и применена соответствующая миграция.
2. В модель Patient добавлено вычисляемое поле full_name с использованием models.GeneratedField (выражение Concat объединяет фамилию и имя на уровне базы данных).
3. В модель Patient добавлено поле slug (URL-идентификатор). Реализован метод save() для автоматической генерации слага и метод get_absolute_url() для формирования красивых ссылок (например, /patients/ivanov-ivan/).

Кастомный менеджер QuerySet:
В файле appointments/models.py создан класс AppointmentQuerySet (наследник models.QuerySet). В нем реализованы методы active(), completed(), cancelled()и for_date(). Менеджер подключен к модели (objects

= AppointmentQuerySet.as_manager()), что позволяет в представлениях использовать Appointment.objects.active() вместо дублирования фильтров.

Работа с ORM и оптимизация запросов:

- В представлении списка приемов (appointments/views.py) используется кастомный менеджер и метод select_related('patient', 'specialist') для предотвращения проблемы N+1 запросов.
- Реализована пагинация (Paginator) с сохранением GET-параметров фильтрации (статуса) при переходе между страницами.
- На странице статистики используется агрегация: aggregate(Count(...)) для подсчета общего количества и values('status').annotate(Count('id')) для группировки по статусам. Также выводится статистика по специалистам с использованием обратной связи annotate(Count('appointments')).

Django Debug Toolbar:
В проект установлен и настроен пакет django-debug-toolbar. В файлах settings.pyи urls.py добавлены необходимые настройки. Панель позволяет анализировать SQL-запросы, выполняемые при загрузке страниц, и подтверждать эффективность использования select_related.

Результат работы

- Административная панель полностью настроена для удобного управления пациентами, врачами и приемами.
- Внедрены статусы TextChoices и кастомный менеджер записей.
- Реализовано вычисляемое поле базы данных (GeneratedField) и слаги для пациентов.
- Все операции с данными выполняются средствами Django ORM. Запросы оптимизированы с помощью select_related.
- Реализована статистика с агрегацией и группировкой.
- Проект проверен с помощью Django Debug Toolbar.
- Обновлен файл README.md, изменения зафиксированы в Git.

Вывод

В ходе практической работы я закрепила навыки работы с административной панелью Django и ORM. Настройка ModelAdmin и Inline позволила создать удобный интерфейс для администратора. Использование TextChoices сделало код более безопасным и читаемым, а кастомный QuerySet — более структурированным. Внедрение GeneratedField и слагов улучшило архитектуру базы данных и удобство URL-адресов. Анализ запросов через Debug Toolbar подтвердил эффективность ORM-запросов в проекте.